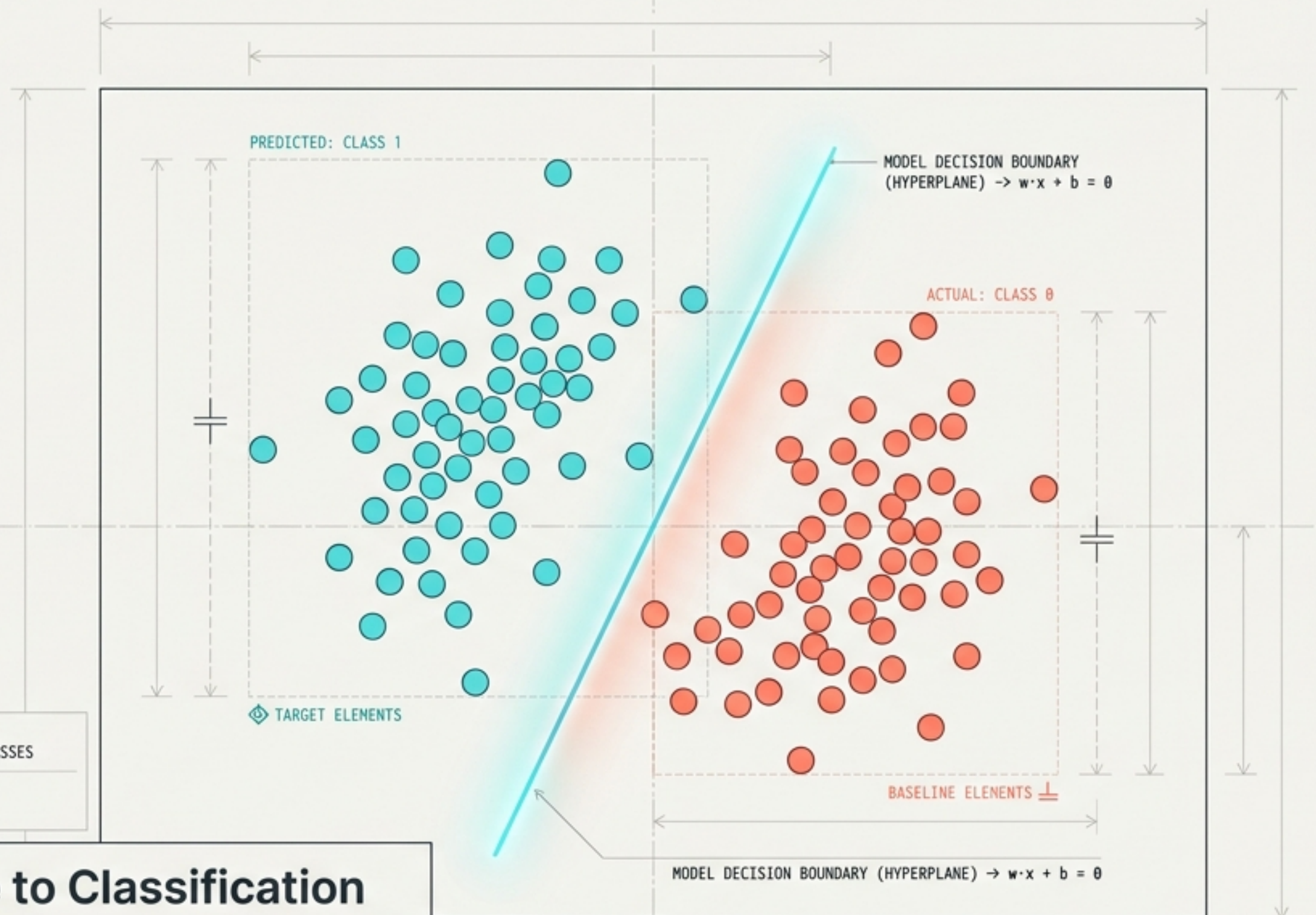


CLASSIFICATION: SUPERVISED LEARNING
GOAL: ASSIGN DATA POINTS TO DISCRETE CLASSES

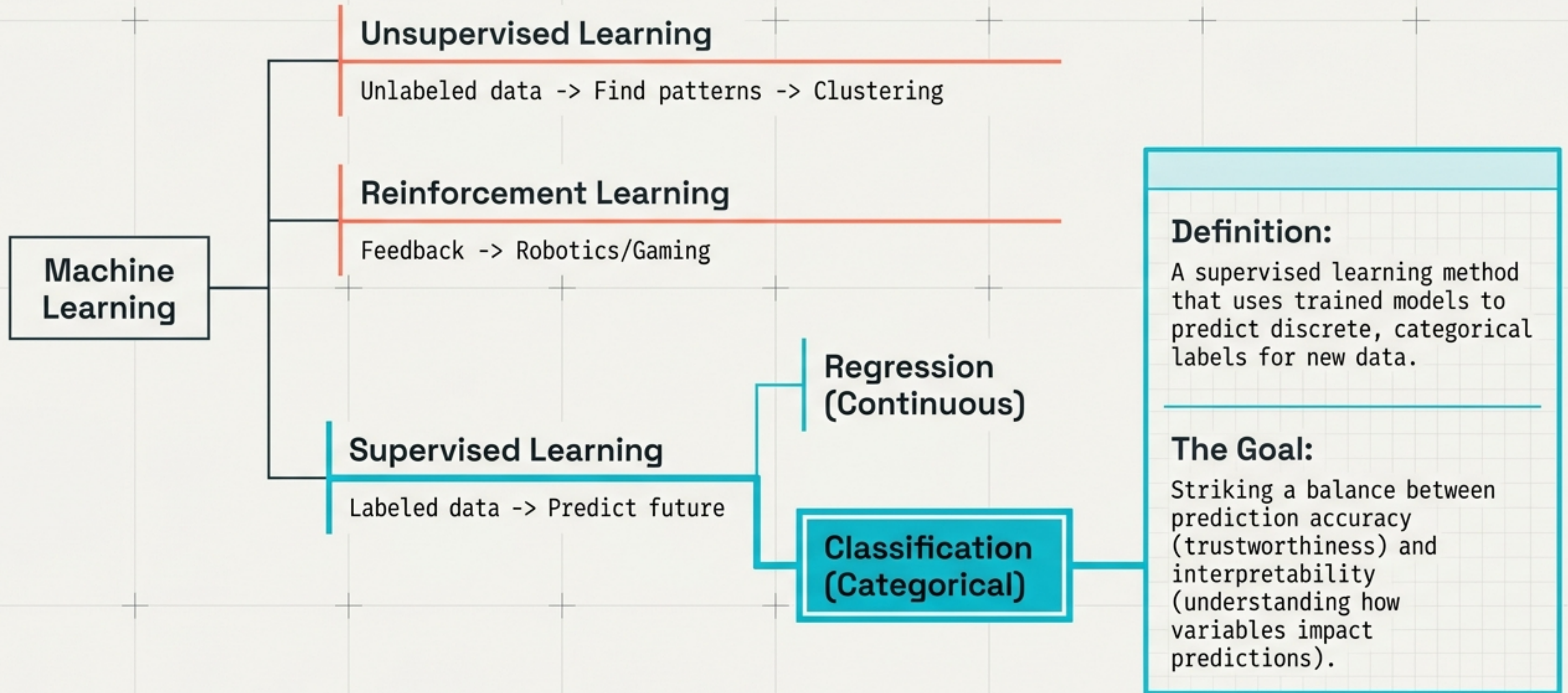
FEATURES: $X = [x_1, x_2, \dots, x_n]$
LABELS: $Y \in \{0, 1\}$

The Field Guide to Classification

A structural blueprint for supervised machine learning, predictive modeling, and algorithm evaluation in Python.



Locating Classification in the ML Landscape



Anatomy of a Prediction: Features vs. Target

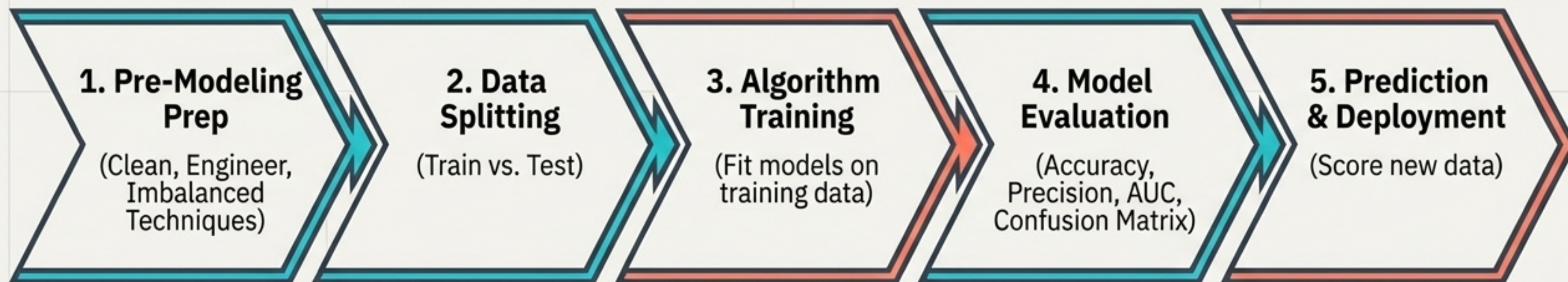
Classification models map the relationship between known inputs (features) and a desired categorical output (target).

Example Case: Predicting whether someone who has seen a website ad will purchase (1) or not purchase (0) based on their demographic data.

Features (X) / Predictors				Target (Y) / The Category
User ID	Gender	Age	EstimatedSalary	Purchased
15624510	Male	19	19000	0
15810944	Male	35	20000	0
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	0
15728773	Male	27	58000	0
15598044	Female	27	84000	0
15694829	Female	32	150000	1
15600575	Male	25	33000	0
15727311	Female	35	65000	0

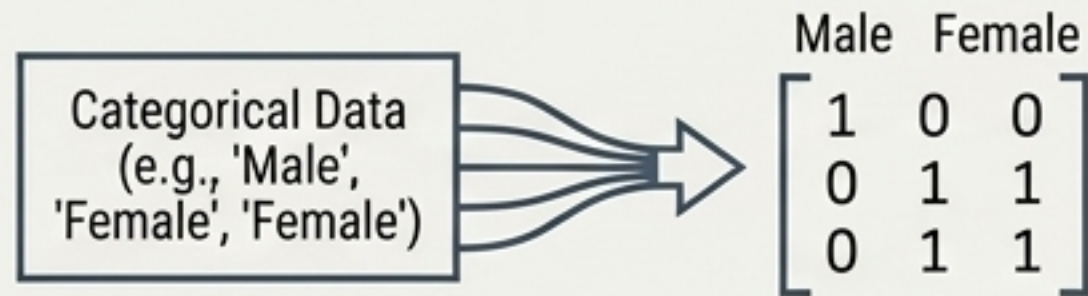
The Classification Modeling Workflow

A structured pipeline transforms raw categorical and numerical data into automated decision-making engines.



Translating Raw Data for Algorithms

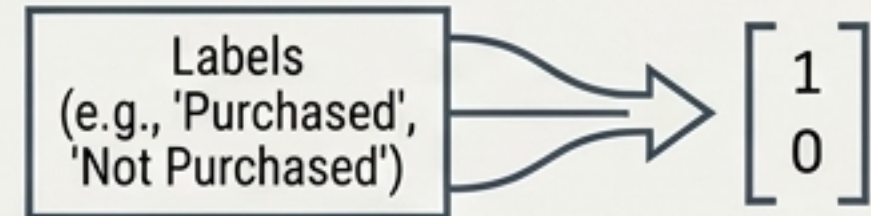
OneHotEncoder



Transforms categorical features into a binary matrix.

```
encoder.fit_transform(categorical_data)
```

LabelEncoder



Encodes string targets/labels into numeric format.

```
encoder.fit_transform(labels)
```

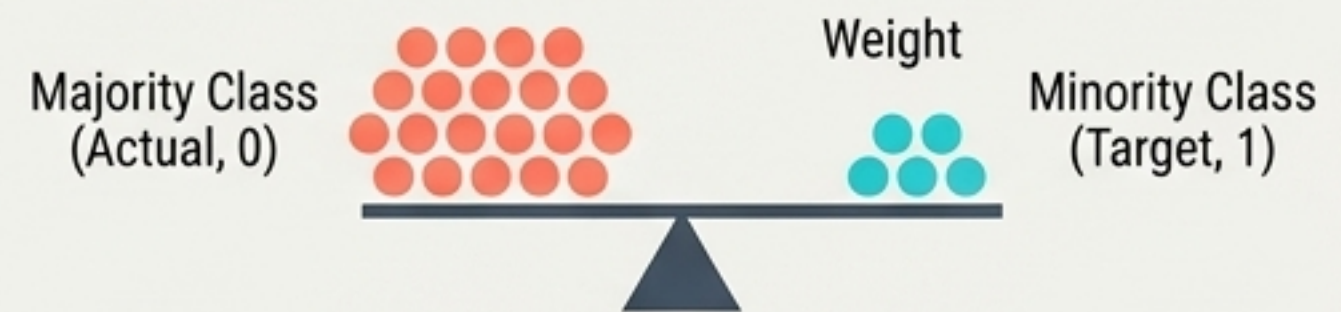
Normalize



Scales features to have zero mean and unit variance (standardization).

```
normalize(data, norm='l2')
```

Class Weights



Computes sample weights to manage imbalanced datasets where one class is rare.

```
compute_sample_weight(class_weight='balanced', y=y)
```


The 80/20 Rule: Training vs. Out-of-Sample Testing

Training Data (In-sample, 70-90%): Used to fit and tune the model's parameters.

Test Data (Out-of-sample, 10-30%): Reserved strictly for realistic estimates of accuracy on unseen data.

```
from sklearn.model_selection import train_test_split

# Assume X (features) and y (target) are prepared dataframes
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 20% for testing
    random_state=42,         # Reproducibility seed
    stratify=y               # Ensure balanced class distribution
)
```

Rows are sampled randomly to ensure representative distribution

The Bias-Variance Tradeoff

“All models are wrong, but some are useful.” — George Box

The goal of training is not perfect memorization (overfitting), but generalization. We tune hyperparameters to find the sweet spot between overly simplistic (bias) and dangerously complex (variance).

High Bias / Underfitting



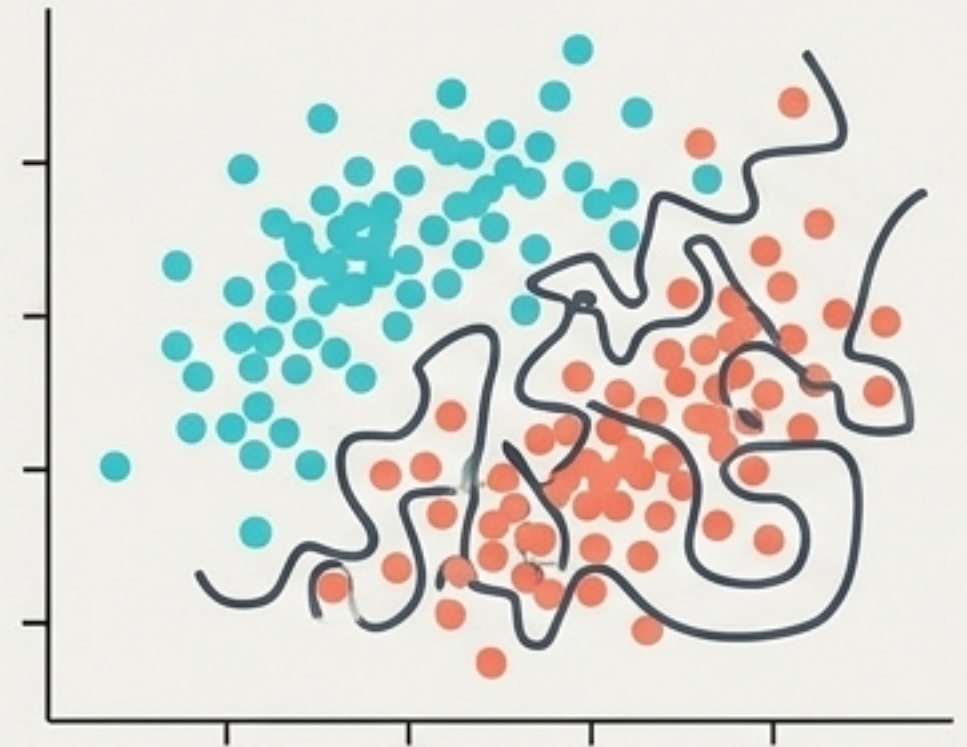
Model is too simple, high error on training data.

Optimal



Balanced model, good generalization.

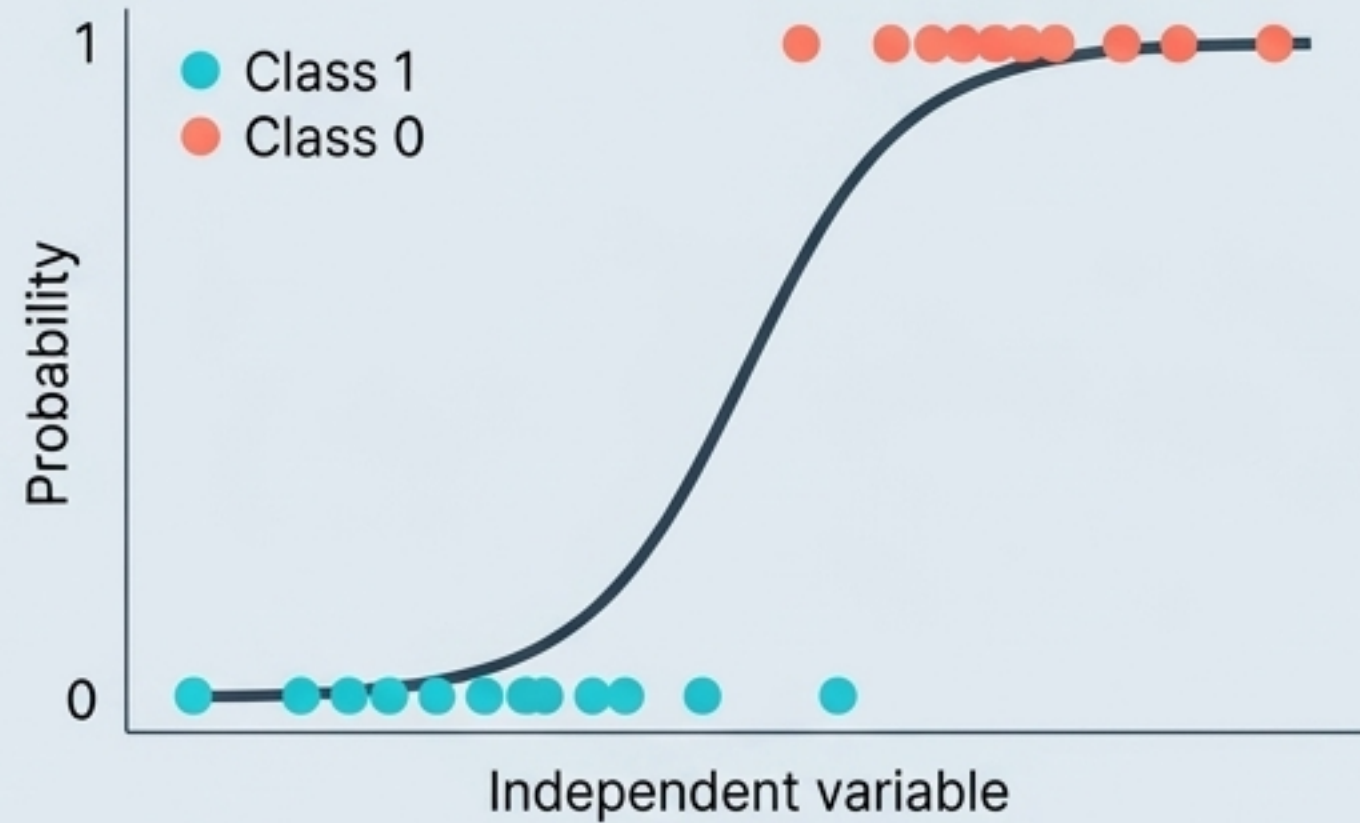
High Variance / Overfitting



Model is too complex, memorizes noise.

Algorithm Taxonomy I: Probability & Distance

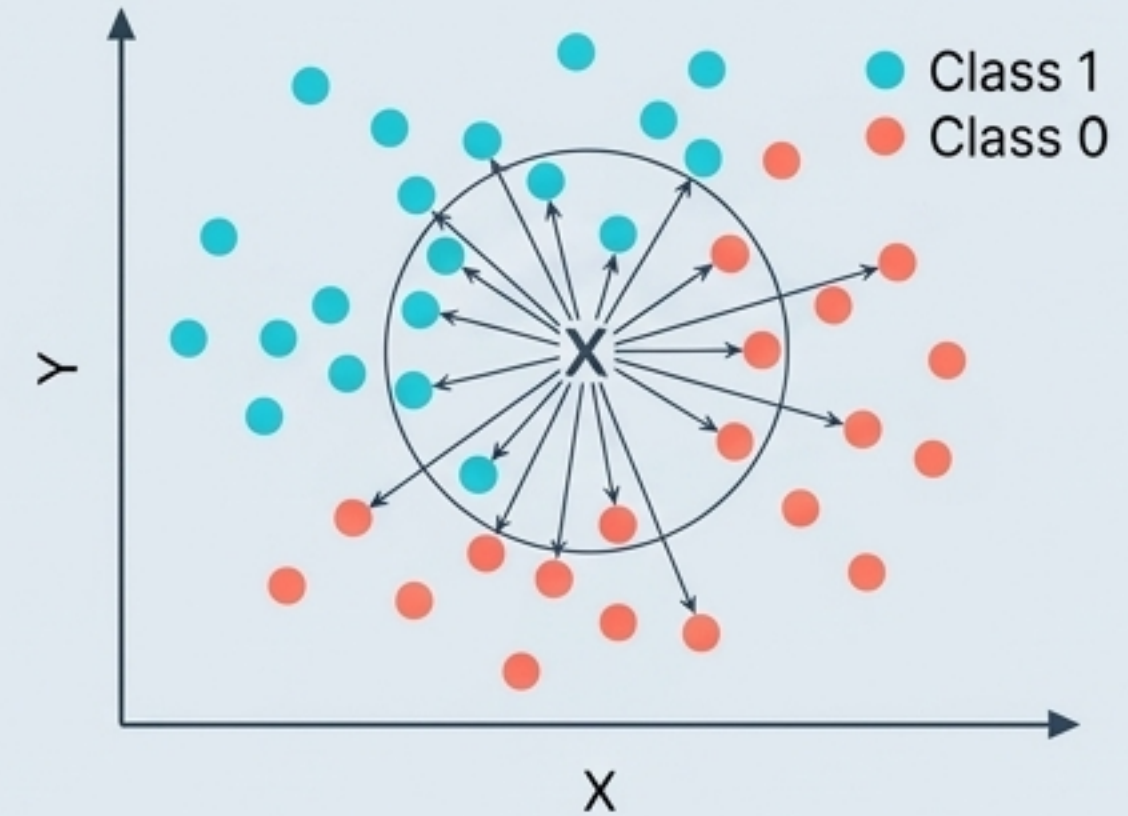
Logistic Regression



Mechanism: Predicts the probability of a class using a logistic curve.

Code: `LogisticRegression()`

K-Nearest Neighbors (KNN)



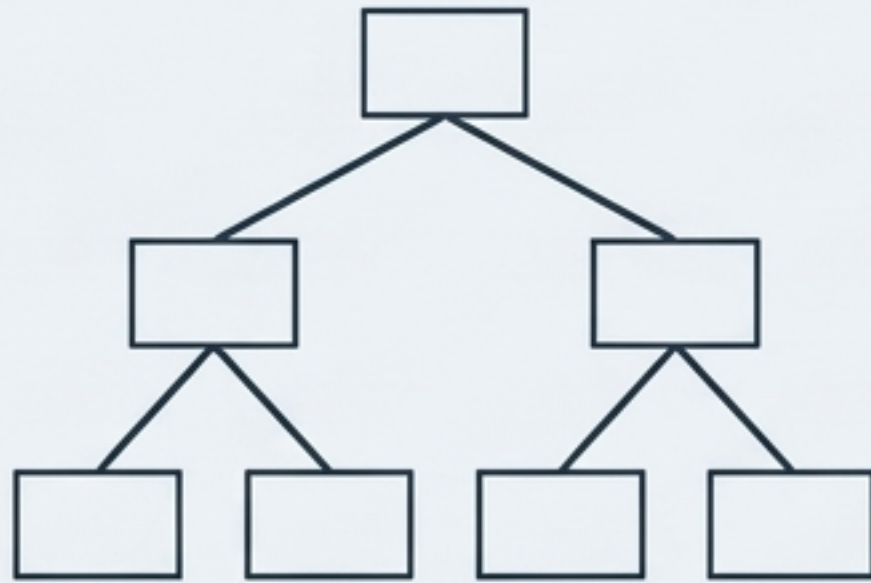
Mechanism: Classifies observations based on the majority class of nearby points.

Hyperparameters: `n_neighbors`, `weights`

Pros/Cons: Simple and effective for small data; computationally expensive as data grows.

Algorithm Taxonomy II: Trees & Ensembles

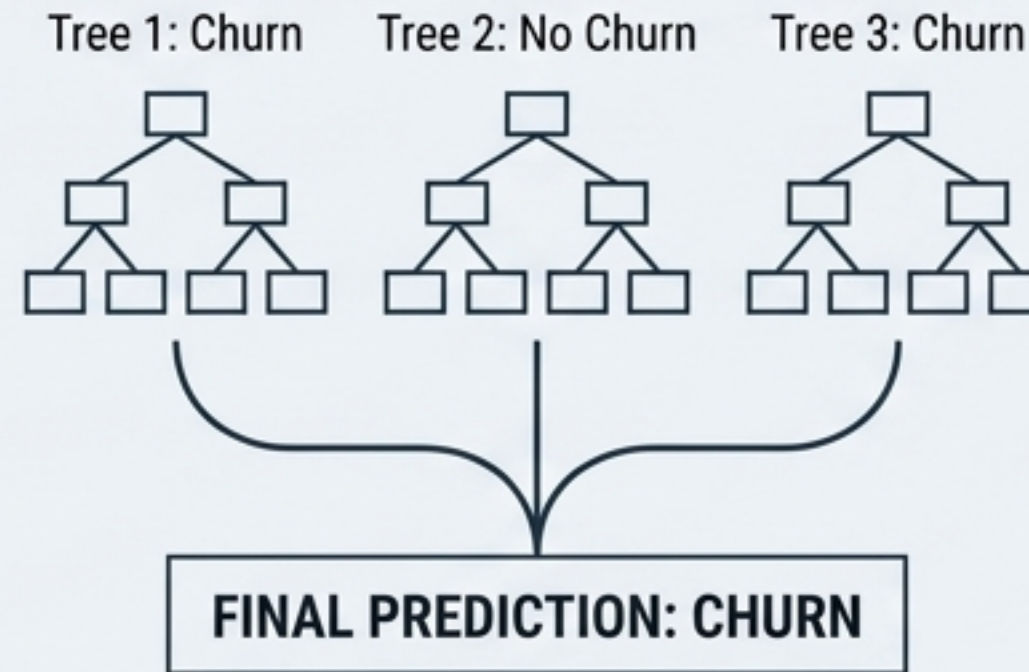
Decision Tree



Splits data recursively to maximize information gain (Gini impurity). Interpretable but prone to overfitting without pruning.

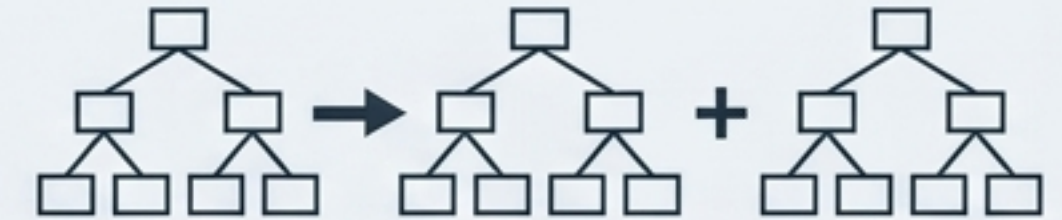
Hyperparameter: `max_depth`

Random Forest



An ensemble method using bagging to train multiple decision trees on bootstrapped data, improving accuracy by reducing variance.

XGBoost

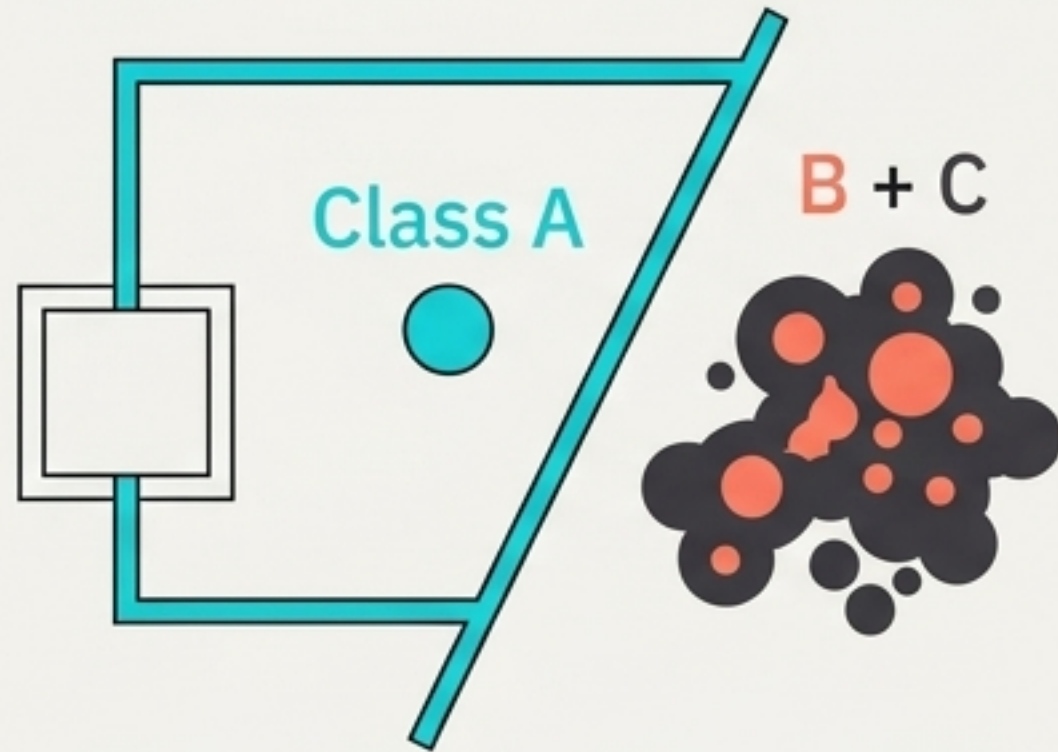


A gradient boosting method building trees sequentially to correct errors from previous trees. Highly accurate but complex to tune.

Hyperparameters:
`learning_rate`, `n_estimators`

Scaling Up: Multi-Class Strategies

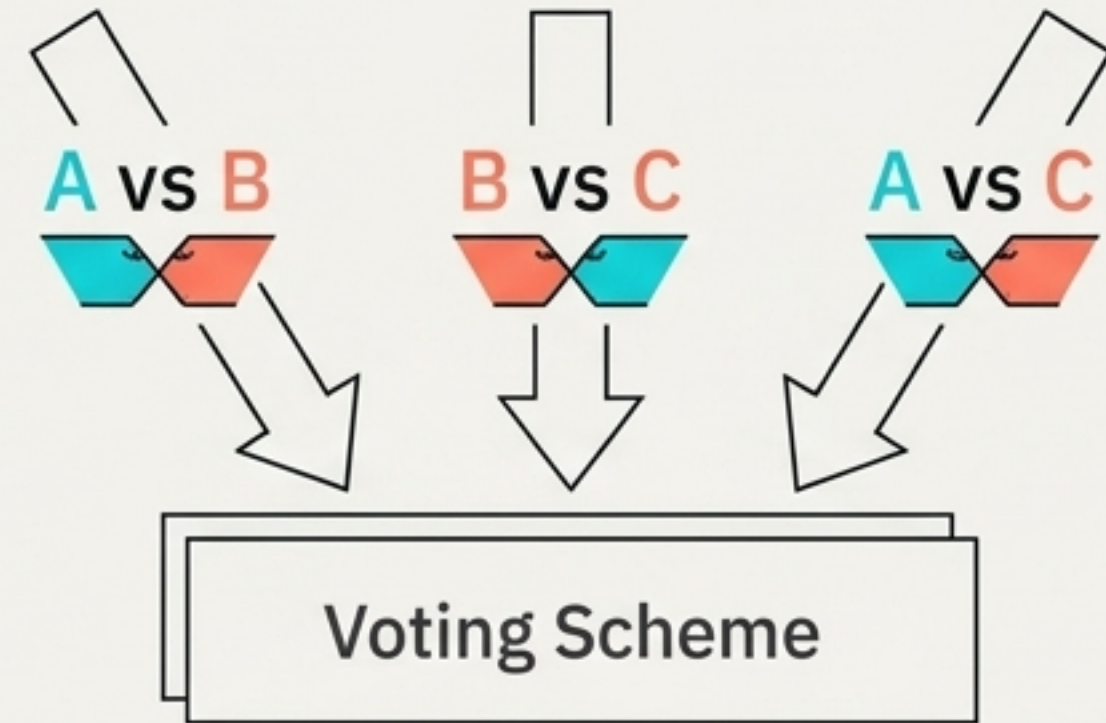
One-vs-All (OvR)



Trains one independent binary classifier per class to distinguish it from the rest. Scalable, but struggles with highly imbalanced classes.

OneVsRestClassifier

One-vs-One (OvO)



Trains $k*(k-1)/2$ classifiers for every pair of classes. The final prediction relies on the most votes. Great for small datasets, computationally expensive for large ones.

OneVsOneClassifier

The Evaluation Diagnostic: The Confusion Matrix

Accuracy alone is dangerous, especially with imbalanced data. The **confusion matrix** reveals where the model is making mistakes by comparing true class counts against predicted class counts.

		Predicted Class	
		Positive (1)	Negative (0)
Actual Class	Positive (1)	True Positive (TP)	False Negative (FN)
	Negative (0)	False Positive (FP)	True Negative (TN)

Decoding the Metrics: Precision, Recall, & ROC-AUC

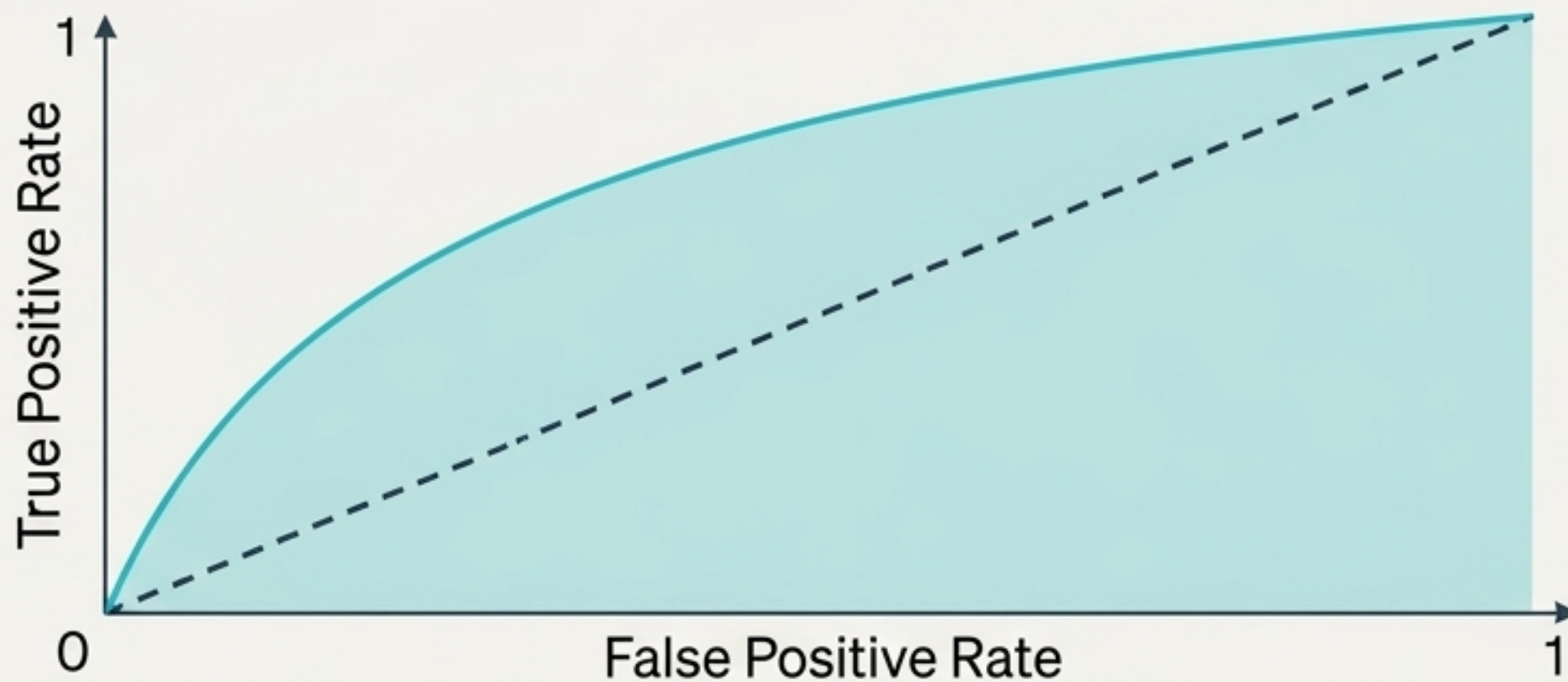
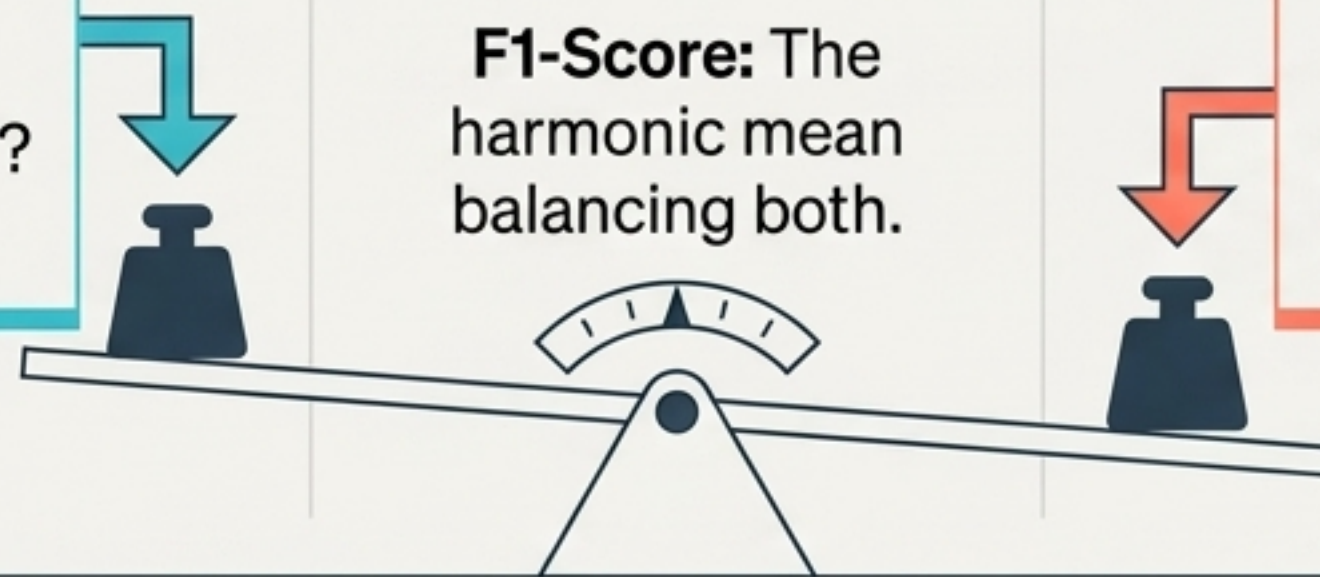
Predicted/Target

Precision: Of all predicted positives, how many were right? Focuses on correctness.

Actual/Baseline

Recall (Sensitivity): Of all actual positives, how many did we find? Focuses on completeness.

F1-Score: The harmonic mean balancing both.



ROC-AUC:

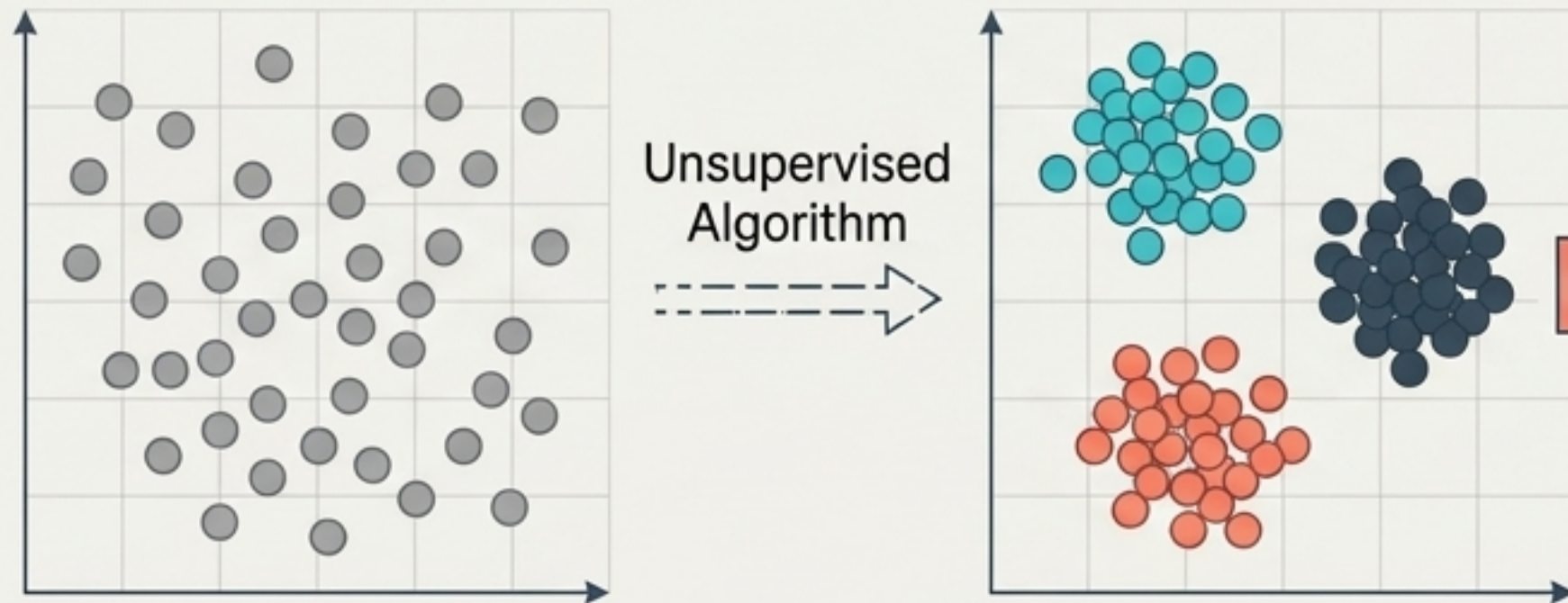
Measures the model's ability to discriminate between classes across all probability thresholds (roc_auc_score).

0.5 is random guessing; 1.0 is perfect.

Bridging Unsupervised and Supervised Learning

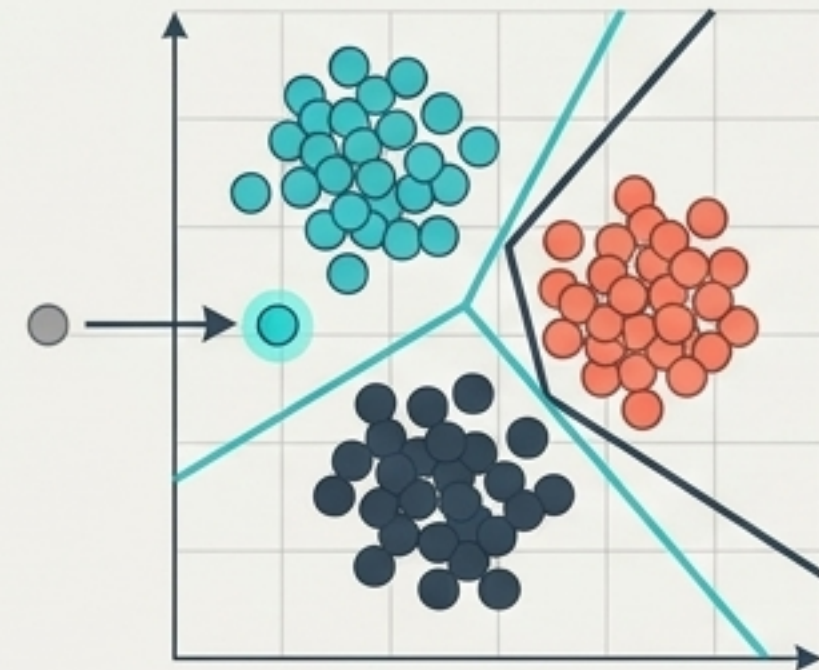
Machine Learning Engine

Clustering (Discovering Segments)



Unsupervised. Used to group customers or data points based on similarities when predefined labels do not exist.

Classification (Predicting Segments)



Supervised. Once clusters are identified and labeled, classification assigns new, unseen data into those known segments based on learned patterns.

Real-World Applications & Impact



Finance (Risk & Retention)

Loan Default & Churn Prediction.

Models output the probability of at-risk accounts based on transaction behavior, enabling targeted retention.



Healthcare (Diagnostics)

Medical Diagnosis.

Training algorithms on cell characteristics with known labels to classify benign vs. malignant tumors.



Natural Language Processing

Text Classification. Using vectorized text data (often with Naïve Bayes or Logistic Regression for <100k rows) to flag suspicious emails as spam or route customer support tickets.

The Complete Implementation Blueprint (Scikit-Learn)

1. Prepare Data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Scale & Split

2. Instantiate and Train Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Learn the Patterns

3. Predict and Evaluate

```
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Score on Unseen Data